

Project Proposal for QIntern 2026

Benchmarking Quantum Algorithms in the Cloud: A Comparative Study of Classical vs. Simulated Quantum Approaches on AWS

Lead Mentor: Joselyn Lagunas

Affiliation: Universidad Virtual del Estado de Guanajuato (UVEG), AWS Community Builder, Mexico

Email: josymonse@gmail.com

Co-Mentor Wanted: Seeking a quantum computing researcher from QWorld

Project Duration: July 1, 2026 – August 15, 2026

1. Project Title

Benchmarking Quantum Algorithms in the Cloud: A Comparative Study of Classical vs. Simulated Quantum Approaches on AWS

2. Project Overview

The transition from classical to quantum programming remains a major challenge for software engineers. While educational proofs of concept exist, there is a lack of systematic, cloud-based benchmarks that quantify exactly how quantum algorithms perform compared to their classical counterparts in terms of execution time, resource consumption, and accuracy when run on today's quantum simulators. This project will bridge that gap by designing and executing a reproducible benchmarking study using Amazon Web Services (AWS), principally Amazon Braket simulators, EC2, and S3.

The intern will implement a set of four problems (random number generation, search, optimization, and quantum key distribution) in both classical Python and with Qiskit/Braket SDK. They will measure and analyse performance metrics under controlled conditions. The final outcome will be an open-source benchmark suite, a technical report, and a short research paper suitable for submission to a quantum computing workshop or conference.

3. Research Questions

- How do quantum algorithms simulated on Braket's local and managed simulators (e.g., SV1) compare to classical implementations in terms of runtime, memory usage, and solution quality for small-scale problem instances?
- What is the overhead introduced by cloud-based quantum simulation, and how does it scale with problem size?
- Can a carefully designed benchmark suite accelerate the training of classical developers in quantum computing by providing immediate, quantitative feedback?

4. Objectives

- Design and implement fair, reproducible benchmarks comparing classical and simulated quantum solutions.
- Leverage AWS infrastructure (EC2, S3, Cloud9, Amazon Braket) to ensure scalability and reproducibility.
- Quantify trade-offs between classical and quantum approaches across multiple problem domains.
- Produce an open-source benchmarking toolkit and a research paper draft.
- Present findings at the QIntern mini-workshop and target a subsequent conference or journal submission.

5. Benchmark Problems and Methodology

The project will investigate four algorithm classes:

- **Random Number Generation:** Compare classical PRNG (e.g., MT19937) with quantum circuit-based RNG running on Braket simulators. Metrics: entropy, statistical randomness test suites (NIST), generation speed.
- **Search Algorithms:** Classical linear search vs. Grover’s algorithm. Metrics: query complexity, wall-clock time for small databases, simulator fidelity.
- **Optimization:** Classical heuristic (e.g., simulated annealing) vs. QAOA implemented via Qiskit/Braket for a chosen MaxCut problem. Metrics: approximation ratio, runtime, resource scaling.
- **Introductory Quantum Cryptography:** Classical Diffie-Hellman key exchange vs. a simulated BB84 QKD protocol. Metrics: key rate, error rate under simulated noise, data overhead.

All experiments will be conducted inside AWS Cloud9 using pre-configured EC2 instances. Results will be logged, stored in S3, and analysed with Python (pandas, matplotlib). The study will carefully document the execution environment to guarantee reproducibility.

6. Technology Stack

- Python, Qiskit, Amazon Braket SDK
- AWS Cloud9, EC2, S3
- Jupyter Notebooks
- Git, GitHub, LaTeX (Overleaf)

7. Six-Week Execution Plan

Week 1: Literature Review and Environment Setup

- Survey existing quantum benchmarking literature and tools (e.g., QED-C, SupermarQ).
- Set up Cloud9 environment with Braket SDK and necessary libraries.
- Define detailed benchmarking protocol and metrics.

Deliverable: Annotated bibliography, configured development environment, measurement framework.

Week 2: Random Number Generation Benchmark

- Implement classical and quantum RNG modules.
- Execute batch experiments, measure randomness quality and speed.

Deliverable: Jupyter notebook with full results, initial plots and statistical report.

Week 3: Search Algorithm Benchmark

- Build Grover’s algorithm and linear search harness.
- Run multiple problem sizes; record query counts and execution times.

Deliverable: Benchmark notebook and scalability analysis.

Week 4: Optimization and Cryptography Benchmarks

- Select MaxCut instances; implement QAOA and classical solver.
- Implement simplified BB84 protocol and classical comparison.

Deliverable: Two notebooks with comparative performance data.

Week 5: Data Analysis and Paper Drafting

- Aggregate results, perform statistical analysis.
- Write first draft of research paper (4–6 pages).

Deliverable: Draft paper and consolidated dataset.

Week 6: Finalisation and Dissemination

- Polish paper, prepare presentation for QIntern mini-workshop.
- Release benchmark toolkit on GitHub with documentation.
- Discuss publication venues (e.g., IEEE Quantum Week, QTEM, QCE workshops).

Deliverable: Final paper preprint, presentation slides, public repository.

8. Deliverables Summary

- Open-source GitHub repository containing full benchmark suite and documentation.
- Four Jupyter notebooks with comparative experiments.
- Technical report / research paper preprint.
- Final presentation for QIntern mini-workshop (16–17 August 2026).

9. Expected Impact

This project will deliver the first open, cloud-based benchmark specifically designed to help classical developers understand the practical trade-offs of near-term quantum algorithms. The dataset and toolkit will enable educators to illustrate quantum advantage (or lack thereof) with real numbers. The resulting publication will contribute to the growing body of work on quantum software engineering and performance evaluation.

10. Future Work

- Extend benchmarks to real quantum hardware when access permits.
- Incorporate additional algorithms (HHL, VQE) and problem domains.
- Develop an automated benchmarking pipeline with CI/CD.
- Conduct user studies to measure educational impact.

11. References

- Michael A. Nielsen and Isaac L. Chuang, *Quantum Computation and Quantum Information*.
- Qiskit Documentation.
- Amazon Braket Documentation.
- Lov K. Grover, “A Fast Quantum Mechanical Algorithm for Database Search”.
- QED-C Benchmarking Suite <https://github.com/SRI-International/QC-App-Oriented-Benchmarks>.